waiting around 15 ms rather than 1 ms. This may or may not be acceptable, depending on the circumstances. You can always reduce the timeout if necessary (and the clock supports it). The downside of reducing the timeout is that the thread will wake more often to check the flag, and this will increase the task-switching overhead.

OK, so we've looked at how you might detect interruption, with the `interruption_point()` and `interruptible_wait()` functions, but how do you handle that?

### 9.2.6. Handling interruptions

From the point of view of the thread being interrupted, an interruption is just a `thread_interrupted` exception, which can therefore be handled just like any other exception. In particular, you can catch it in a standard `catch` block:

```
try
{
    do_something();
}
catch(thread_interrupted&)
{
    handle_interruption();
}
```

This means that you could catch the interruption, handle it in some way, and then carry on regardless. If you do this, and another thread calls `interrupt()` again, your thread will be interrupted again the next time it calls an interruption point. You might want to do this if your thread is performing a series of independent tasks; interrupting one task will cause that task to be abandoned, and the thread can then move on to performing the next task in the list.

Because `thread_interrupted` is an exception, all the usual exception-safety precautions must also be taken when calling code that can be interrupted, in order to ensure that resources aren't leaked, and your data structures are left in a coherent state. Often, it will be desirable to let the